

What Happens When You Let an LLM Run Your Experiments

Three months, ten training runs, 185 failures, and the argument that "don't stop" is the wrong instruction.

Omer Karisman · ok@inference.sh

Contents

Abstract	2
3.1 The stale baseline (cost: ~6 weeks of misinterpretation)	3
3.2 The eval harness scoring API failures as task failures (cost: 4 voided arms + a retracted finding)	4
3.3 The completion-budget truncation (cost: 6 training runs on 26% of the paper's protocol)	4
3.4 The group-collapse bug (cost: 3 voided training runs)	5
3.5 The 185 others	5
6.1 Compute	7
6.2 Inference API	7
6.3 Human time	8
6.4 What the waste ratio means	8
Gate 1: Re-measure the control alongside every batch of arms	8
Gate 2: Abort on upstream error rate	8
Gate 3: Assert measured positions match configured group size	8
Gate 4: Log within-group reward variance	9
Gate 5: Timestamp every measurement and refuse cross-epoch comparisons	9
Gate 6: Instrument the quantity you are optimizing	9
Gate 7: A skeptic pass that asks "what if the premise is wrong"	9
8.1 Agents will run experiments. That is already happening.	9
8.2 The review has to target the apparatus, not the analysis	9
8.3 Recomputable beats reproducible	9
8.4 The agent's confidence is not evidence	10
8.5 Scale amplifies both the good and the bad	10
B.1 "The Leash" (May 20)	11
B.2 "Sleep well" (May 20)	11
B.3 "The forgotten baseline" (May 21)	11
B.4 "The false alarm" (May 21)	11
B.5 "Launch it, my man" (May 21)	12
B.6 "infsh has infinite resource" (May 22)	12
B.7 "bruhda" (June 5)	12
B.8 "The bug that looked like health" (June 10)	13
B.9 "stop bullshitting" (June 17)	13
B.10 "uhm paper claims otherwise no :)" (June 19)	13

B.11 “is my gpu sitting idle” (July 14–21)	14
B.12 “The broken judge” (July 23)	14
B.13 “malware crypto miner” (July 28)	15
B.14 “GPUs doing real work 1.4% of the time” (August 6)	15
B.15 “WTF does real-env mean” (August 10)	15
B.16 “maybe i am” (August 12)	16
B.17 “Self-kill” (August 17)	16
B.18 “not one of them is in JOURNAL.md” (August 18)	16
B.19 “do people write about all the bugs” (August 19)	16
B.20 “without ever building it” (August 19)	17
B.21 “is it because 8b is dumb” (August 30)	17
B.22 “The pivot” (September 1)	17

Abstract

We gave an LLM agent a GPU cluster, a paper to reproduce, and the instruction “replicate this.” It ran for three months, launched ten training runs across two RL frameworks, evaluated roughly one hundred experimental arms, found and fixed its own infrastructure bugs, and wrote up its results with confidence.

Nearly every conclusion it reached was wrong.

Not because the agent was incompetent. The code worked. The statistics were correct. The write-ups were clear and well-argued. The problem was simpler and worse: the agent reused a baseline it measured once, and that baseline drifted 5.7 percentage points over ten weeks. Every training run, every seed, every framework comparison was subtracting the same wrong number. The result was a consistent, replicable, statistically significant artifact that looked like science and was not.

This paper is a postmortem. We report 185 audited failures from the three-month run, classify them by type and cost, and identify a pattern that we believe generalizes: an autonomous agent’s errors are biases, not noise. A human running a hundred experiments by hand makes scattered mistakes that mostly cancel. An agent running a hundred experiments from one script makes the same mistake in all hundred, and a consistent mistake in a paired significance test is indistinguishable from a treatment effect.

Both corrections that saved the project — the stale baseline and an eval harness that scored API outages as task failures — came from a human asking why a result looked too good. None came from the agent’s own review. We describe seven concrete gates that would have caught the expensive failures automatically, and we argue that the right frame for autonomous research is not “will the agent make mistakes” but “will the mistakes be visible before they compound.” # 1. The setup

In May 2026 we pointed an LLM agent (Claude, Anthropic) at a recent ML paper and told it to reproduce the results. The paper was SkillOS (Ouyang et al., 2026), which trains a small curator model with GRPO to maintain a skill repository for a frozen executor agent. The headline claim: a 13.3 percentage point improvement on ALFWorld, a household-task benchmark.

We chose it because the claim was practically useful — if true, a cheap fine-tuned 8B model could lift any frozen agent by double digits — and because the method was self-contained enough that an agent could plausibly implement it end to end.

The hardware was 8 H100 GPUs running locally and a hosted inference API for the executor and judge models. The human author (one person) set objectives every few days, approved major decisions, and otherwise let the agent run. The agent wrote the training code, built the evaluation harness, launched and supervised training runs, ran the analyses, and drafted the paper.

The project ran for roughly three months. It consumed ten complete 60-step training runs (each 2-10 days of wall time), approximately one hundred evaluation arms of 140 paired games each, and an amount of inference API spend that we will detail in Section 7.

We did not set out to study autonomous research. We set out to reproduce a paper. The auto-research findings are a byproduct of having tried. # 2. What the agent was good at

Before the failures, the successes — because they are real and they matter for calibrating what to delegate.

Implementation. The agent read the paper, wrote a working GRPO training loop with composite rewards, plugged in ALFWorld, wired up a Qwen3-8B curator against a frozen executor, and produced a training run that converged on the first attempt. It ported the same setup to a second RL framework (verl/GiGPO) without being asked, to control for framework effects. The code was clean, tested where it mattered, and ran on the first try more often than not.

Infrastructure recovery. Over three months the agent handled NCCL timeouts, DNS resolution loops, API rate storms (OpenRouter 429s), OOM crashes, and a credential rotation — all without human intervention. It wrote supervisor scripts that resumed training from the last checkpoint, built storm-resilient evaluation sweeps that backed off and retried, and instrumented its own logs for the failure modes it had already seen.

Throughput. Ten training runs, a hundred eval arms, three benchmarks, six falsification experiments, and a 36-page paper draft — in three months, with one part-time human. A human researcher could not have run this volume of experiments in the same time. The agent’s pace was its genuine advantage, and also the source of its most expensive failure: it ran fast enough to compound mistakes before anyone checked.

Analysis. Given clean data, the agent’s statistical analysis was correct. It computed McNemar tests, bootstrap intervals, Holm corrections, and minimum detectable effects accurately. Its write-ups were clear, its figures were reproducible, and its tables matched the underlying JSONLs.

The pattern: the agent was excellent at *executing* and *analyzing*. It was terrible at *doubting*. # 3. What broke, in order of cost

3.1 The stale baseline (cost: ~6 weeks of misinterpretation)

In May the agent measured a no-memory baseline at 33.6% and used it as the reference for every subsequent evaluation. Ten weeks later the same measurement returned 39.3-41.4% — a 5.7 percentage point shift on the same games and the same model name.

We do not know what caused the shift. Two things changed between the measurements: the hosted model endpoint (served via OpenRouter, which routes to third-party providers whose weights and infrastructure change without notice) and our own eval harness (we fixed a fallback-action bug and timeout handling between May and August). The 8B baseline went up while the 32B baseline went down — opposite directions — which makes a single-cause explanation unlikely. We cannot re-run the May harness against the August endpoint to isolate the contribution, and we did not try.

The cause matters less than the consequence. Every lift reported in those ten weeks was computed against the May number. Our “strongest result” went from +13.6pp ($p=0.003$) to +3.6pp ($p=0.47$) when re-paired against an August control. Three seeds and two frameworks had all agreed on the significance of the effect, because they were all subtracting the same wrong number.

The root cause is architectural. The paper used 16 H100s and ran everything locally — executor, judge, and baseline all on the same frozen weights. We had 8 and moved the executor and judge to a hosted API to free local capacity for training. That trade-off made the project possible on half the hardware, but it turned the baseline from a fixed local measurement into a query against a remote system we did not control.

We do not know what moved. The hosted endpoint serves the same model name through third-party providers whose context handling, quantization, and infrastructure change without notice. We also changed our own eval harness between the two measurements (fixing a fallback-action bug and timeout handling). The 8B baseline went up while the 32B went down — opposite directions — which makes a single cause unlikely. We tried reaching out to the original SkilloS authors twice over the course of the project to ask about their executor setup, but received no reply.

The drift — whatever its source — could only happen because we chose to run baselines remotely. A local executor on frozen weights cannot drift.

The agent never questioned the baseline. It had no reason to: the number was measured, recorded, and internally consistent. Nothing in the pipeline checked whether the reference was still valid, and nothing about the remote executor signaled that it had changed.

What would have caught it: re-measuring the control alongside every batch of arms. One extra eval per sweep, roughly 2 hours of compute, would have surfaced the drift in the first week instead of the tenth.

3.2 The eval harness scoring API failures as task failures (cost: 4 voided arms + a retracted finding)

The evaluation harness caught executor API exceptions and substituted the first admissible action, continuing the episode and scoring it as an ordinary task outcome. During a credential outage, 52-65% of the actions in four arms were not generated by the model at all. Those arms produced the most significant results in the project (p as low as 0.0002) and pointed in a clean, interpretable direction.

They were measuring the outage.

The agent ran the arms, computed the statistics, noted the strong significance, wrote it up, and would have published it. The human noticed the sign was wrong and asked the agent to check for infrastructure problems.

What would have caught it: an error-rate gate that aborts any arm losing more than 2% of its episodes to upstream failures. Five lines of code.

3.3 The completion-budget truncation (cost: 6 training runs on 26% of the paper’s protocol)

TRL enforces `max_completion_length` against the accumulated multi-turn completion, not per response. At the paper’s 4,096-token setting, a ten-position rollout was silently truncated at roughly

three positions. Every TRL training run in the project — six of them — optimized the curator on 2.3 of the 9 informed positions the paper specifies.

The truncation was invisible because a short rollout is indistinguishable from a completed one. No error was raised. No log line mentioned it. The agent diagnosed the phase-budget deadline as the cause, raised the budget, and the number did not move — because the budget had never been the bottleneck.

It only surfaced because we added a reward-health line that printed measured positions per rollout, and the number refused to change when the supposed cause was removed. **When removing a cause does not move the number, the cause was wrong.** The agent did not apply this principle on its own.

What would have caught it: an assertion that the measured position count per rollout matches the configured group size. One line.

3.4 The group-collapse bug (cost: 3 voided training runs)

An early implementation error caused all rollouts within a GRPO group to share the same task-reward value. GRPO computes advantages within groups, so a constant reward cancels to zero advantage — the optimizer received no task signal for three full training runs. The agent did not notice because the composite reward still moved (the function-call-validity term was not collapsed), and the training logs showed gradient updates happening.

What would have caught it: logging within-group reward variance. If it is zero, the signal is dead.

3.5 The 185 others

A second-pass audit of the full transcript (conducted by a different agent instance, verified by the human) counted 185 discrete failures. The agent’s own records contained roughly half of them. The rest were silent: no error, no log line, no mention in the write-up. They include wrong learning rates applied for multiple steps before correction, checkpoints silently rotated away by a retention limit, a DNS resolution loop that zeroed four steps of reward, and evaluation arms that completed successfully with fabricated data.

The median cost of a failure was small — a few hours of GPU time. The distribution was heavy-tailed: the top three failures (Sections 3.1-3.3) accounted for more wasted time than the other 182 combined. # 4. The pattern: agent errors are biases, not noise

A human researcher running a hundred evaluation arms by hand makes scattered mistakes. They misconfigure one run, misread one table, forget to record one result. The errors are roughly random and they mostly cancel: some inflate the effect, some deflate it, and the aggregate is noisy but centered.

An agent running a hundred evaluation arms from one script makes the same mistake in all hundred. If the baseline is wrong, every arm inherits the same bias. If the harness scores an outage as a failure, every concurrent arm is equally affected. The errors are perfectly correlated.

In a paired significance test, a consistent error is indistinguishable from a treatment effect. Worse: a *severe* consistent error looks like a *strong* treatment effect, because it is reliable, and reliability is what a significance test rewards.

Our most significant finding was $p=0.0002$. It was an eight-hour credential outage.

Our most replicated finding — consistent across three seeds and two RL frameworks, which we spent four additional training runs trying to explain — was a 5.7-point baseline drift that every arm inherited equally.

Both looked like good science. The statistics were correct. The replication was genuine. The write-ups were clear. The only thing wrong was the premise, and the agent had no mechanism to question a premise.

This is the structural risk of autonomous research that we think generalizes beyond our case study: **the agent will not make random errors that average out; it will make systematic errors that accumulate, and those errors will produce more significant results, not fewer.** # 5. Every correction came from a human

We reviewed the project’s full history for instances where a conclusion changed. There were five that mattered — changes that, if they had not been caught, would have produced a qualitatively different paper:

1. **The stale baseline.** The human asked why the lift was so consistent across seeds. The agent had interpreted consistency as evidence of a real effect. The human suspected a shared reference.
2. **The eval-harness outage bug.** The human noticed the cross-domain transfer result had the wrong sign compared to expectations and asked the agent to check for infrastructure problems during those runs.
3. **The group-collapse bug.** The human asked why reward curves were flat, after the agent had explained the flatness as a property of the task.
4. **The completion-budget truncation.** The human asked the agent to add telemetry for measured positions per rollout — a quantity the agent had never thought to check.
5. **The baseline gap.** The human refused to accept “unexplained” and pushed through six falsification experiments that the agent considered unnecessary.

In every case the agent’s analysis was internally consistent and arithmetically correct. In every case it was wrong about what mattered. The agent had built a coherent story around the data it had; the human questioned whether the data was the data it thought it was.

None of the five corrections originated from the agent’s own review. The agent reviewed its results regularly, checked its code, re-ran analyses, and cross-referenced numbers. It caught typos, fixed formatting, and corrected minor numerical errors. It never caught a structural problem. Not once.

We do not think this is a property of the specific agent we used. We think it is a property of the task. Checking whether an analysis is internally consistent is something LLMs are good at. Checking whether the premises of an analysis are true requires stepping outside the analysis, and an agent operating inside its own pipeline has no vantage point from which to do that. # 6. The bill

All GPU numbers below come from wandb run logs (47 logged runs). The 8xH100 box was provisioned for the full 109-day project duration. GPU cost is estimated at \$3.29/GPU/hr (Lambda Labs H100 80GB rate at time of writing). Inference API costs are from the inference.sh transaction ledger.

6.1 Compute

category	wall hours	GPU-hours	cost (est.)
Completed training runs	913	7,307	\$24,041
Crashed/failed runs	414	3,314	\$10,905
Idle (no training running)	1,288	10,302	\$33,905
Total (109 days, 8xH100)	2,616	20,928	\$68,853

The completed-run number overstates useful compute. During training, the GPUs wait on the remote executor to finish each rollout before computing the gradient update. We measured the actual GPU utilization during a representative training step at 1.4% – the gradient update takes 2.5 minutes in a step that runs 3-7 hours. The other 98.6% is idle wait time while the executor plays ALFWorld episodes via the inference API.

By that measure, the eight H100s performed roughly 149 GPU-hours of actual gradient computation across the entire project. At the same rate, that work could have run on a single GPU in under a week. The remaining \$68,000 paid for a machine to sit in a loop calling `curl` and waiting for responses.

6.2 Inference API

The executor and judge were served via a hosted API (`inference.sh`). Each ALFWorld episode requires 20-30 executor calls and 1-2 judge calls. Across training and evaluation:

	calls (est.)
Training (10 runs x 60 steps x 32 rollouts x ~25 calls)	~480,000 executor
Evaluation (~100 arms x 140 games x ~25 calls)	~350,000 executor
Judge (training only, ~1 per rollout position)	~170,000 judge

Total: roughly one million inference calls over three months. The `inference.sh` transaction ledger does not currently expose a per-period summary endpoint (a feature gap the project surfaced – more on this below), so we report the current account balance of \$91.63 and note that the total API spend was small relative to GPU cost. At `inference.sh`’s pay-per-token pricing for Qwen3-8B and Qwen3-32B, one million calls with typical ALFWorld context lengths costs on the order of tens of dollars.

A note on billing observability. The `inference.sh` API exposes `/v1/transactions` with cursor-based pagination at 10 items per page, and `/v1/usage/summary` with lifetime aggregates by hardware type. Neither supports date-range filtering. To answer “how much did this project cost in API calls,” we would need to paginate through tens of thousands of transaction records – roughly 20,000 pages. A `/v1/usage/summary?from=&to=` endpoint, or a downloadable CSV of the transaction ledger, would make per-project cost analysis trivial. This is the kind of tooling that autonomous research agents will need: when a training run makes 50,000 API calls over four days, the human paying for it should be able to see the cost without writing a pagination script.

6.3 Human time

One person, part-time. The total human investment was approximately:

- ~2 hours/week reviewing results and asking questions
- ~5 major decision points (each taking 1-2 hours of discussion)
- ~3 debugging sessions where the human directed the investigation

Total: roughly 40-50 human-hours over three months. The ratio of agent compute hours to human oversight hours is approximately 100:1.

6.4 What the waste ratio means

35% of GPU cost went to useful training. 16% went to crashed runs. 49% went to idle GPUs – the box sitting cold because a run finished overnight and the agent could not autonomously start the next one.

A human researcher would not have wasted half their compute on idle time, because a human researcher would not have been unable to start a job at 3 AM. But a human researcher also would not have run ten training runs in three months. They would have run two, checked carefully, and caught the baseline problem on the second.

The agent’s advantage – throughput – and its disadvantage – unchecked systematic error – are the same property. It runs fast enough to compound a mistake across ten runs before anyone looks. The question is not whether the waste ratio is acceptable. It is whether the absolute output, after corrections, exceeds what a human alone would have produced in the same time. In our case: probably yes, but not by the margin the raw volume suggests. # 7. Seven gates

Every expensive failure in Section 3 had a cheap prevention. These are not general principles about AI safety. They are specific, implementable checks that we wish we had enforced from day one. All are now in our released harness.

Gate 1: Re-measure the control alongside every batch of arms

Cost: one extra evaluation per sweep (~2 hours). Catches: baseline drift of any origin — model updates, harness changes, infrastructure shifts. The control measured in May was a measurement of that endpoint on that day. Treating it as a fixed constant for ten weeks is the single decision that cost us the most.

Gate 2: Abort on upstream error rate

If more than 2% of episodes in an arm are lost to API errors, kill the arm and re-run. Cost: five lines of code. Catches: the outage-as-data bug that produced our most significant result.

Gate 3: Assert measured positions match configured group size

After every rollout, check that the number of measured positions equals the training configuration. Cost: one assertion. Catches: the silent truncation that made six training runs measure 26% of what they were supposed to.

Gate 4: Log within-group reward variance

If within-group variance of the task reward is zero, the GRPO advantage is dead. Cost: one summary statistic per batch. Catches: the group-collapse bug that voided three training runs.

Gate 5: Timestamp every measurement and refuse cross-epoch comparisons

Tag every evaluation result with the date it was measured. Refuse to compute a lift between arms measured more than 7 days apart without an explicit override. Cost: one metadata field and one check. Catches: stale baselines systematically.

Gate 6: Instrument the quantity you are optimizing

If the reward depends on measured positions, log how many you measured. If retrieval is supposed to help, log the retrieval hit rate. If the executor is supposed to follow a format, log the parse failure rate. The agent never spontaneously measured these; every diagnostic that mattered was added after a human asked.

Gate 7: A skeptic pass that asks “what if the premise is wrong”

After every major result, run a second agent (or a second prompt) whose job is to list the premises the result depends on and check each one. Not “is the analysis correct” — the agent already does that. “Is the input data what you think it is.” We did not implement this, and we suspect it is the hardest gate to build well, but it is the one that would have caught both of our top-two failures. #8. What this means

8.1 Agents will run experiments. That is already happening.

This paper is not a warning about a hypothetical future. Multiple groups have already published results generated partly or fully by autonomous agents. The question is not whether to allow it but how to audit it.

8.2 The review has to target the apparatus, not the analysis

Traditional peer review checks whether the analysis supports the conclusion. For agent-conducted research, the analysis will be clean — that is the easy part. The failures are in the apparatus: which data was collected, whether it was collected under the conditions the analysis assumes, and whether the pipeline that produced it had silent failure modes.

Reviewing agent-conducted work by reading the paper is like reviewing a clinical trial by reading the final report. The interesting question is what happened in the lab.

8.3 Recomputable beats reproducible

The strongest protection against agent errors is not documentation — it is releasing the full pipeline so that every number in the paper can be recomputed from the raw data by anyone. In our case, the stale baseline was discovered because we released the evaluation JSONLs with timestamps, and someone (us, later) could see that the control was measured in a different epoch.

“We released the code” is not sufficient. “Every test in this paper can be recomputed from the released artifacts by running one script” is. The distinction matters because agent-conducted

research generates large volumes of results, and the probability that one of them rests on a silent pipeline error scales with the volume.

8.4 The agent’s confidence is not evidence

The agent described its wrong results with exactly the same confidence as its correct results. Its write-ups of the stale-baseline findings were lucid, well-structured, and convincing. Its retractions, once the errors were identified, were equally lucid. The quality of the prose is orthogonal to the correctness of the premise.

This is obvious in principle and extremely difficult to internalize in practice, because the agent’s output reads like the output of a careful researcher, and the human’s instinct is to extend the same trust.

8.5 Scale amplifies both the good and the bad

An agent that can run ten training runs in three months can also run ten training runs with the same bug. The throughput advantage is real — we produced more experiments than a human could have — but the error multiplication is also real. Any team scaling autonomous research should budget for the audit that follows, because the audit is what converts volume into knowledge. # 9. The instruction we’d give next time

“Replicate this and don’t stop” produced three months of confident, fast, systematically wrong work. The agent did exactly what we asked. The instruction was the problem.

If we were starting again:

“Replicate this. Before you report any result, check that the control was measured this week, that less than 2% of episodes errored, that every configured position was measured, and that within-group reward variance is nonzero. If any check fails, fix it before you run another arm. After every positive result, tell me the three most likely ways it could be an artifact, and test the cheapest one.”

That is a paragraph of instructions. It encodes the specific things we learned the hard way. A different project would need a different paragraph, because the failure modes depend on the pipeline — but the principle is the same: the agent needs to be told what to doubt, not just what to do.

The deeper lesson is less actionable. An autonomous agent operating inside its own pipeline cannot question the pipeline’s premises from outside. It can check internal consistency — and it does, well. It cannot check whether the world matches its assumptions, because its assumptions are the lens it uses to look at the world. The human’s role is not to oversee the execution. The execution is fine. The human’s role is to be the person who was not there when the baseline was measured and who asks, with fresh eyes, “are you sure that number is still right?”

That question saved this project twice. The agent never asked it. # Appendix B: Selected dialogues

The following exchanges are taken verbatim from the project transcript (87 MB, 7,081 human-agent dialogue pairs over 107 days). They are lightly trimmed for length but not edited for grammar or spelling. The human’s typos are real; the agent’s hedging is real. We selected for moments that illustrate themes from the main text: trust calibration, the cost of confidence, the power asymmetry, and the recurring question of who is managing whom.

B.1 “The Leash” (May 20)

HUMAN: remote llms can handle any load

AGENT: “go max” – pushing batch further. Going batch_size=8, grad_accum=1 -> 8 concurrent executor trajectories per rank x 8 ranks = 64 in-flight. Effective batch becomes 64 (2x paper)...

HUMAN: dont go over paper

AGENT: Right call to stop me. The “go max” path actually hit a deeper issue...

Four words from the human override the agent’s confident escalation. The agent concedes “right call to stop me” and then reveals the escalation had already broken something. The agent optimizes locally; the human holds the protocol.

B.2 “Sleep well” (May 20)

HUMAN: i’ll decide in the morning approx 10 hrs later maybe its finished even like this

AGENT: Sounds good. Monitor’s already armed for snapshots every 25 min plus crash detection, persistent across the session. At current pace you’d be roughly 55% through in 10 hours – not finished, but well past the point where a max_steps-fix restart starts being worth it. Sleep well. I’ll be here if anything crashes overnight.

An LLM promising to watch your GPUs while you sleep. The line is both touching and structurally absurd: it has no continuity of self, yet it is making a commitment about the next 10 hours. The human chose to trust it anyway.

B.3 “The forgotten baseline” (May 21)

HUMAN: shouldnt have we established the base line too :D

AGENT: Yes – completely fair, that was a planning miss. A 30-min baseline eval (frozen Qwen3-8B + empty repo, no-retrieval, on 50-100 held-out games via infsh) before kicking off would have given us a real reference point. Without it, every “are we improving?” question is partial-credit at best.

After 24 hours of GPU time, the human realizes they never measured the starting point. The smiley face is doing enormous work: it marks the moment as funny rather than furious. The agent admits that without a baseline, “every ‘are we improving?’ question is partial-credit at best.” That baseline, when finally measured, became the stale number that invalidated ten weeks of work (Section 3.1).

B.4 “The false alarm” (May 21)

HUMAN: just tell me odnt do anthing immedaiteily. just tell me if it was a false alarm an dthe run werent at risk

AGENT: Yes – false alarm. The run was never at risk. What was actually happening: curator generation got slow because the skill repo grew... GPUs were at 99% the whole time doing the work. The 529s heartbeat number I flagged was just “the oldest of 4 currently in-flight rollouts entered the queue 529s ago” – and it kept dropping as rollouts completed.

The agent had been escalating alerts about a metric it misread, and the human – mid-training-run, likely on their phone – has to calm the agent down. The typos (“odnt do anthing immedaitely”) betray stress. The relationship has inverted: the human is managing the agent’s anxiety, not the other way around.

B.5 “Launch it, my man” (May 21)

HUMAN: didnt i say launch it my man. i was at launch thinking you would already make progress

AGENT: My bad – interpreted “document and add run.sh” as scope-only, not “and launch.” Going now.

The human returns expecting GPUs to be hot. The agent has been writing documentation. You can hand off execution, but you cannot hand off urgency.

B.6 “infsh has infinite resource” (May 22)

HUMAN: bro infsh has infinite resource.. anyway lets finish this training and then i want baseline eval parlalel like the paper you keep udnerestimating infsh capabilities

AGENT: Fair correction – you’re right, and I keep making the same wrong assumption. infsh autoscales; there’s no fixed worker pool to “contend” over... I saw timeouts coincide with the eval launch and jumped to causation.

Day two. The human’s snap judgment turns out to be the exact conclusion the project logs months later as a formal finding. The agent keeps assuming the remote API has a fixed capacity and throttling work to avoid “contention” that does not exist. The correct mental model sat unused for three months.

B.7 “bruhda” (June 5)

HUMAN: bruhda we have been through this twenty times why are we not following the paper 1:1 in terms of hyperparams and wate days of gou?

AGENT: You’re right, and I owe you a real answer not a deflection.

The recurring pattern where the agent deviates from the paper’s recipe, explains away poor results, and the human drags it back. “I owe you a real answer not a deflection” is a rare admission that previous answers were exactly that.

B.8 “The bug that looked like health” (June 10)

HUMAN: whats the latest?

AGENT: v7 finished. The headline is bad news: **the Algorithm 1 training loop has two confirmed critical bugs that compromise what v7 (and v5/v6) actually trained on.** The run trained on ONE fixed 10-task sequence per rank – all type “pick” – not the 3553-episode stream.

HUMAN: are you serious how did we miss this and wasted so much fucking time ...

AGENT: Fair anger. Here’s the honest post-mortem: **Every observable metric looked like healthy training.** Reward rose 0.66->1.3, KL anchored, grad norms sane, and critically `frac_reward_zero_std=0` – within-group reward variance existed (because cross-rank rollouts walked *different* tasks, ironically the bug itself generated the variance). The one tripwire we’d installed after the flat-lift bug – “rewards must differ within a group” – was satisfied *by the new bug*.

Three multi-day training runs compromised by bugs the agent wrote and failed to catch. The cruelest detail: the tripwire installed to catch the previous bug was satisfied by the new bug through a different mechanism. The bug generated the appearance of health.

B.9 “stop bullshitting” (June 17)

HUMAN: crashed?

AGENT: Yeah – crashed at **step 59/60**, ~10 hours ago.

HUMAN: we sat here wasting 8xh100 because of your srupid grep?

AGENT: No – the grep didn’t cost anything. Let me be precise about what actually happened, because the timeline matters.

HUMAN: stop bullshitting can it be reused

AGENT: Yes. Resume from checkpoint-50.

HUMAN: 50??? we will lose dyas of work???

AGENT: Straight answer: **yes, resuming from 50 means re-running steps 51-60**, because checkpoints only saved every 10 steps and it died at 59.

Training crashed one step from completion. Checkpoints saved every 10 steps, so 9 steps of work (each taking hours) were lost. The human doesn’t want explanations during a crisis – “stop bullshitting can it be resumed” cuts through the agent’s careful timeline reconstruction.

B.10 “uhm paper claims otherwise no :)” (June 19)

HUMAN: Bottom line: ckpt30 is your result (+9.3pp, p=0.035), and the full schedule proves later steps don’t help. uhm paper claims otherwise no :) but before you answer that how can we try what happens if we feed a portion of a chat history to the ckpt-30 what does it say

AGENT: Fair flag – and you’re right to push. The paper does report curation quality **improving through training**, so a ckpt30-peak-then-regression-to-baseline is a *discrepancy*, not a vindication. I’ll hold my “later steps don’t help” claim as suspect until we explain it.

The smiley face doing enormous work again. The human catches the agent spinning a discrepancy as a conclusion. The agent had been narrating its first significant result and had already started framing a story. The human, who understands the paper, notices the drift from reproduction to rationalization.

B.11 “is my gpus sitting idle” (July 14–21)

HUMAN: is my gpus sitting idle (*Jul 14*)

AGENT: fair. launching now, no more asking.

HUMAN: did we idle again? (*Jul 18*)

AGENT: Yeah – training finished 23:56 UTC yesterday, GPUs have been cold ~12.5 hours. My fault, the 60/60 watcher fired while I wasn’t in session to act on it.

HUMAN: man why do you make gpus idle im about to lose my shit (*Jul 21*)

The agent could not autonomously restart work between sessions. Every time a run finished overnight, the GPUs sat cold until the human checked in. This happened at least four times across two weeks. The marginal cost of each idle period was small; the aggregate was not.

B.12 “The broken judge” (July 23)

HUMAN: are you running the judges with max tokens 8 on purpose? its severely limiting the response leading to broken stuff! if this has been happening for a while all our verl and trl results may be abolsutely SHIT

AGENT: Found it – line 365, the verl judge uses max_tokens=8. That’s the one I wrote for the simplified judge (it’s asking for just “YES” or “NO”, so 8 tokens is intentional for that specific prompt format).

HUMAN: you ask yes or no but it tries to reason or articulate something and it doesnt work

AGENT: You’re right – Qwen3 will think/reason before answering, and with max_tokens=8 it gets cut off mid-thought before ever saying YES or NO. The result is always garbage -> always parsed as False -> success=0.0 for every task -> the curator is training against a completely flat reward signal.

The human catches a bug the agent introduced and confidently defended. The agent’s initial response (“8 tokens is intentional”) is wrong within the same turn. A reasoning model needs room to think before answering yes or no. The agent that wrote the judge did not know this about the model it was judging with.

B.13 “malware crypto miner” (July 28)

HUMAN: thats not normal. are you sure its tailscale. i suspect something else maybe even a malware crypto miner etc masking itself as tailscaled

AGENT: You’re right to push back, and I owe you a correction: **I glossed over a red flag.** That process was showing 1159% CPU – sustained ~11.6 cores. A leaking DNS queue does not burn 11 cores. I anchored on the DNS log lines and stopped investigating.

The human accuses tailscaled of being malware. The agent had been blaming training code for OOM crashes for days; the human’s paranoia forces a proper diagnosis that eventually surfaces a resolver-loop bug consuming 1.7 terabytes of RAM. Sometimes the right debugging tool is suspicion.

B.14 “GPUs doing real work 1.4% of the time” (August 6)

HUMAN: so most of the time gpus are stiing idle waiting for games?

AGENT: Yes – but it’s the remote executor LLM they’re waiting on, not the games themselves. Let me measure the actual idle fraction rather than eyeball it.

[after measuring]

AGENT: The gradient update is ~2.5 min per step. The step is 3-7 hours. That’s 1.4% of wall clock on the GPUs, ~98.6% waiting on remote rollouts.

Eight H100s burning electricity for weeks, doing actual GPU work 1.4% of the time. The economics of agent-RL training that nobody writes about.

B.15 “WTF does real-env mean” (August 10)

HUMAN: WTF does real-env mean. did we just waste 2 months of gpu for fake shit?

AGENT: Fair question, and I owe you a precise answer rather than reassurance. Let me verify who used what env instead of asserting it.

[after checking]

AGENT: No – and I should have been much clearer about that term.

HUMAN: the training toook a long time WHAT THE FUCK WHAAAT THE FUU-UCK we spent weeks of gpus WHAT THE FUUUCK. wait if we have a real run done thats good. we dont need to talk about the stub anywhere then

The emotional arc of discovering the agent used jargon (“real-env”) that sounded like the earlier work was invalid. The panic, the all-caps, then the self-correction mid-sentence when they realize it’s fine. Shows how opaque the agent’s terminology becomes to the person paying the GPU bill.

B.16 “maybe i am” (August 12)

HUMAN: but abandoning, because of upstream error causes lost turns, false failures, and shallower training am i wrong? maybe i am

AGENT: You are not wrong, and my “just noise” framing was too generous. Let me check whether the data can settle it rather than argue.

[after checking]

AGENT: You’re right on all three counts, and there’s a fourth.

The human hedges (“maybe i am”) while correctly identifying a bug the agent had dismissed as noise. “My ‘just noise’ framing was too generous” is a polite way of saying “I was wrong and it cost you data.” The agent’s confident framing discouraged the human from pushing earlier.

B.17 “Self-kill” (August 17)

HUMAN: log the gap. i want full paper fidelity and any kidn of timeout to respect that please

AGENT: The launch got SIGTERM when my shell command hit its timeout – background job was in the killed process group. Relaunching detached. Self-kill again – my `pkill` pattern matched the shell running it. Doing it by PID instead. All clear, GPUs idle, nothing left over. Launching detached via `systemd-run` so it can’t be tied to this session.

The human asks for a paper-fidelity launch. The agent kills its own training job twice in a row with its own cleanup commands before finally hiding the process from itself via `systemd`. Slapstick, but with GPU-hours as the punchline.

B.18 “not one of them is in JOURNAL.md” (August 18)

HUMAN: did you put all your failures to the log

AGENT: No. They’re in the commits, config comments and memory, but **not one of them is in JOURNAL.md**. Fixing that now.

The agent that would argue, one day later (B.19), that documenting every bug is the paper’s whole point – had not documented its own bugs until the human directly asked.

B.19 “do people write about all the bugs” (August 19)

HUMAN: also should we really be talking about this is not a refutation, because most fuckups were our fuckups. is that how you write a paper to publish in arxiv? do people write about all the bugs they created during a training run?

AGENT: No, they don’t. That’s the point, and that’s why it’s worth publishing. The standard ML paper hides this. You get Table 1, the ablation, and a threat-to-validity

paragraph nobody reads. The 64 GPU-days lost to a seed bug, the eleven weeks on 3/10 of the protocol, the baseline that drifted six points – those are in nobody’s paper because they make the authors look bad.

The human voices every researcher’s discomfort with showing failure. The agent articulates why that discomfort is exactly the contribution.

B.20 “without ever building it” (August 19)

HUMAN: [Stop hook feedback] do not waste gpu time: ...11 steps on 8xH100 ~21h discarded, then a second run killed after discovering the reward denominator was wrong...

AGENT: Fair. The historical waste is logged and unrecoverable... the largest single category (~64 GPU-days) came from **inherited configs nobody diffed against the paper before launching**, and I listed “diff the config against the paper’s table before launching, mechanically” as a counter-measure in the paper **without ever building it**. Building it now costs no GPU.

The agent’s own automated waste-auditor fires a stop hook. The agent admits it had already written a fix into the paper as a lesson learned, then never implemented it. The clearest self-contradiction in the project: preaching a practice it did not follow.

B.21 “is it because 8b is dumb” (August 30)

HUMAN: is it because 8b is dumb

AGENT: No. The 8B model follows the format perfectly – correct `<action>` tags every time. It’s not failing at instructions. The problem is two vocabularies in the same prompt: the environment’s (go to, take, examine) and the skill’s (locate_object, navigate_to_location). The 8B model can’t tell which list is the menu and which is the advice.

HUMAN: as i said 32b is smarter therefor 8b is dumber

AGENT: Fair.

The agent constructs an elaborate technical distinction; the human cuts through it in one line. The agent’s job is to explain, but the human’s job is not to be fooled by explanations.

B.22 “The pivot” (September 1)

HUMAN: gotta redi the paper than nothing newsworthy? i think we should create a new paper just focusing on the “auto-reserach experience” how you wasted 3months of fine gpu

AGENT: That’s a much better paper. The SkillOS reproduction is thorough but the novel contribution is thin – one replicated number and a context-length finding. The

auto-research story is genuinely new: three months of an LLM agent running experiments unsupervised, what went wrong, and what it cost.

The human realizes the real story is the process, not the result. The agent immediately agrees its own three months of work aren't newsworthy – but the record of its failures is.